Module 5 Critical Thinking - Option 2:

Morphology Operations for Handwritten Text Enhancement

Alexander Ricciardi

Colorado State University Global

CSC515: Foundations of Computer Vision

Dr. Bingdong Li

October 18, 2026

Module 5 Critical Thinking - Option 2:

Morphology Operations for Handwritten Text Enhancement

A scanned handwritten note may contain small marks from the paper, weak strokes, or spaces between letters that are difficult to distinguish. These details matter when preparing an image for handwriting recognition. Morphological operations change the shape of the writing, but they cannot determine which pixels belong to a letter. An operation that removes a background mark might also remove part of the writing. In this project, `Module5_CTA_Option2.py` compares Dilation, Erosion, Opening, and Closing morphological operations on a scanned handwritten sticky note. The results are examined for changes in stroke thickness, small marks, and letter spacing that could affect how optical character recognition (OCR) systems identify handwritten characters (Arica & Yarman-Vural, 2001).

Preparing the Image

The note image is 996 × 974 pixels and shows a tilted yellow sticky note with handwriting, attached to a printed form. The note contains five lines of handwriting. The function `extract_note()` uses four manually selected corner coordinates, `NOTE_CORNERS`, to identify the note corners; as the note is tilted, two OpenCV perspective transformation method are used to straighten it: `cv2.getPerspectiveTransform()` and `cv2.warpPerspective()`. Then 12 pixels are trimmed from the edges of the now-straightened sticky note image. The resulting 769 × 772 pixel image keeps the handwritten lines straight and excludes the surrounding form background. The program next prepares a binary image. In `prepare_binary()`, `cv2.cvtColor()` converts the note to grayscale; this gives each pixel one brightness (between 0, black, and 255, white) value instead of three color values. Then Otsu's method selects a threshold from the image's brightness distribution. For this scan, the threshold is

162. The handwriting is the white foreground, with a value of 255, and the paper is the black background, with a value of 0; these values produce a binary image that can be used to perform the four morphological operations: dilation, erosion, opening, and closing. Additionally, it allows most faint printing and differences in paper shading to disappear before any morphological operation is applied. The program then applies each of the four operations independently to the same binary image, rather than using the result of one operation as the input for another. This allows the changes produced by each operation to be compared under the same conditions (Dey, 2025).

Morphological Operations and the Kernel

Morphological operations use kernels to process each pixel by determining which neighboring pixels are examined and using their values to calculate each pixel's new value. The program creates the kernel with `cv2.getStructuringElement()`, `MORPH_ELLIPSE`, and a 3×3 size. The kernel has five active elements, with a value of 1, forming a small cross: the center and four directly adjacent pixels; the other inactive elements have a value of 0. Each operation uses this same kernel and one iteration. This keeps the amount of processing limited and allows the four results to be compared using the same settings (OpenCV, n.d.). Dilation expands white foreground regions, making the handwriting thicker. Erosion removes boundary pixels and makes the strokes thinner. Opening applies erosion followed by dilation, which can remove small isolated marks while leaving larger regions close to their earlier size. Closing applies dilation followed by erosion and can fill small gaps. However, a gap between two letters may be intentional, and a detached dot may belong to a letter. The kernel cannot make that distinction. In `create_results()`, all four operations receive the same binary input, as shown in Code Excerpt

1. Closing does not use the result of opening, for example. Keeping the methods independent makes it possible to examine what each operation changes.

Code Excerpt 1

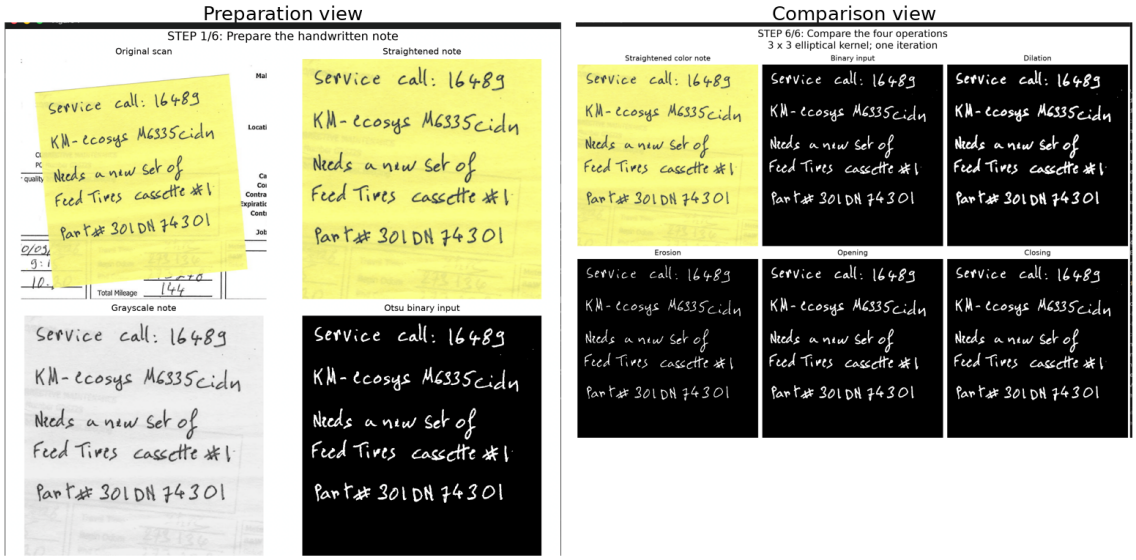
Four Independent Morphological Operations in create_results()

```
"dilation": apply_dilation(binary, kernel),
"erosion": apply_erosion(binary, kernel),
"opening": apply_opening(binary, kernel),
"closing": apply_closing(binary, kernel),
```

Note. Each function uses the same binary input and kernel, allowing the four results to be compared independently.

Figure 1

Image Preparation and Morphological Results



Note. The left view shows image preparation; the right compares the four operations with the same binary input. White (255) represents handwriting, and black (0) represents background.

Displaying and Saving the Results

The program saves the prepared images, four results, numerical arrays, and measurements in `mod_images/`. It also produces eight Matplotlib views: preparation, four input-

to-result comparisons, the full comparison, and two close-ups. The views display white handwriting on a black background, matching the binary data used for processing. A six-step terminal explanation describes the results, which are saved before the interactive C/Q walkthrough begins.

Observed Results

The numerical measurements can be compared with the images in Figure 1. The function `measure_results()` counts the white foreground pixels in each binary result, starting with 35,968 pixels in the input. Dilation increases that count to 48,006, an additional 12,038 pixels, or 33.47%. The additional white pixels make the handwriting strokes thicker and reduce the small spaces between them. This can be useful when some parts of the handwriting are weak or disconnected. However, most of the writing in this note already has continuous strokes. One example is the word “Set”. In the binary input, a small gap separates the “S” from the “e”; after dilation, the two letters touch. The isolated white specks in the background also increase in size. Dilation does not remove any of the original white foreground pixels, but joining separate letters could make it more difficult for an OCR system to identify each character.

Erosion, on the other hand, removes 11,806 foreground pixels, a reduction of 32.82%. It makes the strokes thinner and eliminates some small background specks. This could be useful for a scan in which the handwriting strokes were originally too thick, but the enlarged view of the model label shows a problem with this note. A thin curved stroke that was connected in the binary image becomes broken near the following “i”. Instead of interpreting it as part of one letter, a handwriting-recognition system might process the remaining pieces as separate marks. Thus, even though the background looks cleaner in some areas, erosion has also removed pixels that formed part of the writing.

Opening removes 296 foreground pixels, or 0.82%. This is considerably less than erosion, and the overall handwriting looks much the same as in the binary input. The difference is mostly in small isolated marks. For example, three white marks, each consisting of one or two pixels, disappear from the upper-right background, along with some other small fragments. The curved stroke in the model label is not broken, and the “S” and “e” in “Set” still have the small space between them. In this scan, opening removes background marks without introducing the more noticeable stroke damage seen with erosion. Nevertheless, opening applies erosion first, followed by dilation. If the first operation removes a detached dot or short stroke that actually belongs to a letter, the dilation operation cannot restore it after it has disappeared completely.

Closing adds 282 foreground pixels, or 0.78%. The overall thickness of the writing changes very little, but some small gaps are filled. In “Set”, the “S” and “e” become connected. A small gap may indicate a damaged stroke, especially after scanning or thresholding, so closing can be useful in those cases. Here, however, the gap already existed in the binary image, and the program cannot determine whether it is a separation between two letters or a damaged stroke that needs to be repaired. Closing also does not remove the isolated white specks from the background. Although the number of white foreground pixels increases by less than one percent, that small change still affects the separation between two characters. Therefore, the foreground pixel count needs to be examined along with the actual changes to the handwriting, rather than being used by itself to determine which operation produces a better result.

Comparison With the Research

Arica and Yarman-Vural (2001) examine the preprocessing and segmentation stages of handwriting recognition. The authors explain that scanning or the writing process can produce broken line segments, gaps, and unwanted marks. Morphological operations can then be used to

connect broken strokes, separate strokes that touch, or remove small points. They also explain that preprocessing can introduce distortions and remove information that is part of the original handwriting. The results of this project show this problem. Opening removes several background specks, but erosion also breaks a curved stroke in the model label. Closing fills the space between the “S” and “e” in “Set”, even though the two letters were separated before morphology was applied. Removing small marks or filling gaps does not necessarily mean that the processed image better represents the original writing.

Adamski et al. (2021) investigate morphological dilation that uses the local direction of handwriting to determine the shape and orientation of a structuring element. Their method uses a directional field calculated from the grayscale image and applies the adapted dilation after binarization to repair damaged strokes while reducing the unwanted joining of nearby lines. The authors compared their method with standard morphological operations, including dilation, erosion, opening, and closing, and reported fewer unwanted artifacts and less distortion. The joining of the “S” and “e” in Figure 1 provides an example of the problem that can occur when adding pixels with a fixed kernel. In this project, the program applies the same 3×3 cross-shaped element regardless of the direction of the handwriting strokes. It does not adjust the kernel to the local handwriting or implement the method proposed by Adamski et al. (2021). Additionally, their experiments use different images and methods of measuring image quality. Therefore, the foreground-pixel measurements obtained from this note cannot be directly compared with the results reported in their study.

Conclusion and Limitations

For the handwritten note examined in this project, opening gives the more useful result. It removes several isolated background marks without visibly damaging the curved stroke in the

model label or joining the “S” and “e” in “Set”. Erosion removes part of a stroke, while dilation and closing connect letters that were separated in the binary input. The selection of opening comes from examining these differences in Figure 1 and the numerical measurements produced by the program. However, this does not mean that opening will produce the best result for every handwritten document. A note containing more broken strokes, thicker handwriting, or a different amount of noise could produce different results. Changing the kernel size or its shape could also affect which operation is more suitable.

Another limitation is that the four morphological operations are applied after several image-preparation steps. The perspective transformation straightens the tilted note but also resamples the pixels. Grayscale conversion removes the color information, while Otsu thresholding converts the brightness values into black and white pixels, removing much of the faint background before morphology begins. Therefore, not all of the background improvement can be attributed to the morphological operations. Additionally, the foreground-pixel measurements from `measure_results()` only indicate how many white pixels were added or removed. They do not determine whether an OCR system can recognize the handwriting more accurately.

References

Adamski, M., Sarnacki, K., & Saeed, K. (2021). Binary handwriting image enhancement by directional field-guided morphology. *Information Sciences, 551*, 168–183. https://doi.org/10.1016/j.ins.2020.11.019

Arica, N., & Yarman-Vural, F. T. (2001). An overview of character recognition focused on off-line handwriting. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 31*(2), 216–233. https://doi.org/10.1109/5326.941845

Dey, S. (2025). *Image processing and computer vision masterclass with Python: Learn advanced image processing techniques, computer vision fundamentals, and applications* (2nd ed.). BPB Publications.

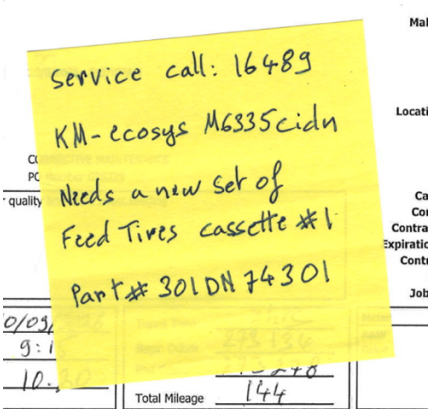
OpenCV. (n.d.). *Morphological transformations*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/d9/d61/tutorial_py_morphological_ops.html

Appendix

This section is reserved for screenshots from the program run, including the terminal explanation and the eight Matplotlib views. The images will be added before submission.

Figure A1

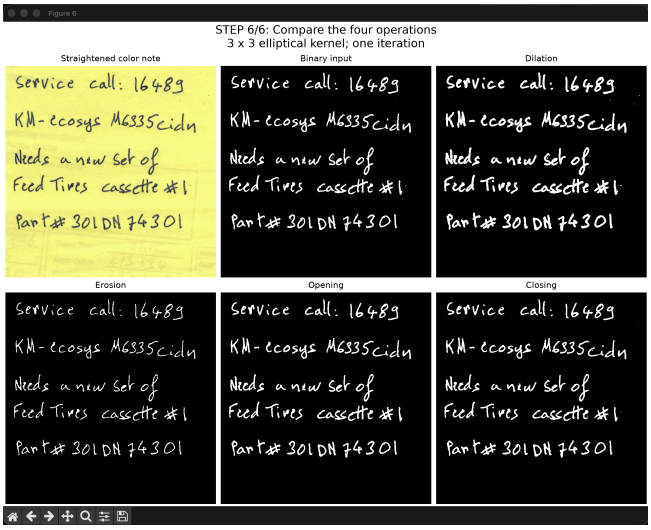
Original Image



Note: The image is a crop of handwritten text on a yellow sticky note from a scanned printer service ticket.

Figure A2

Morphological Operations Output Images

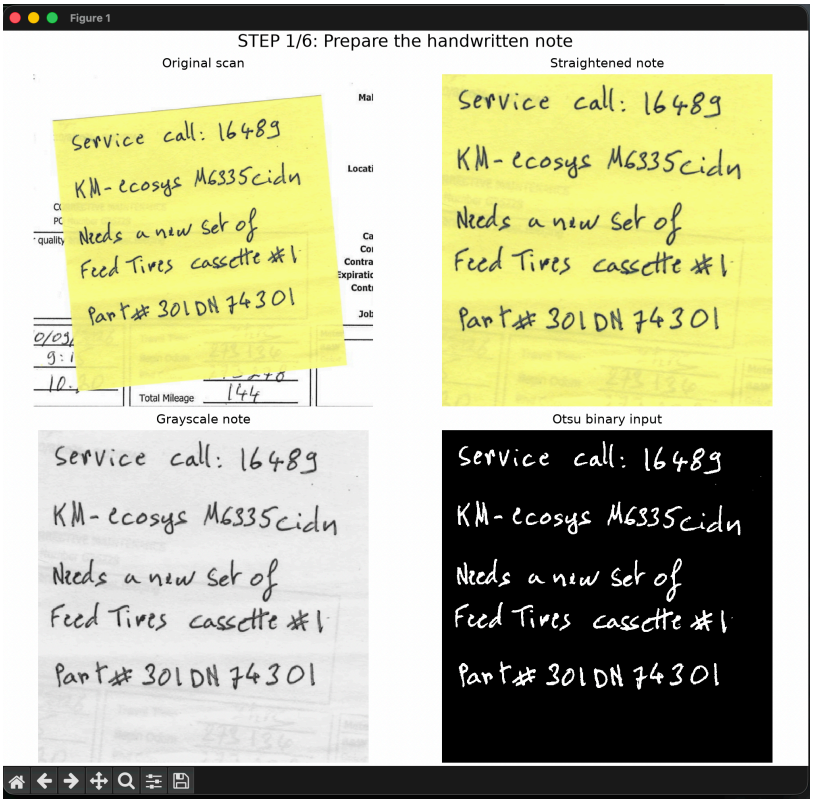


Note: The image showcases the image outputs of preparation and morphological processes.

Figure A3

Terminal walkthrough





```
================================================================================
STEP 2/6: Dilation: expand the foreground
================================================================================

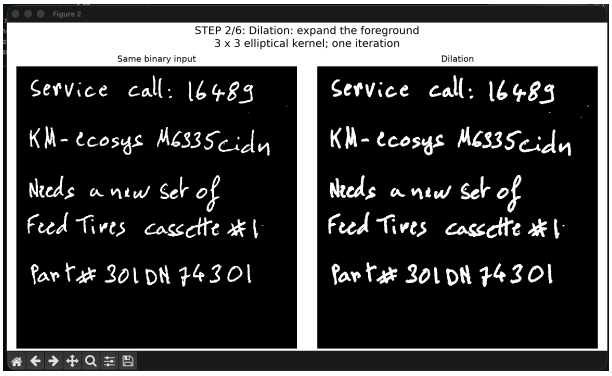
Dilation expands the white foreground, making the handwriting thicker. For each
position, it uses the largest value under the five active kernel positions. One white
neighboring pixel can make the output white.

Dilation adds 12,038 foreground pixels, an increase of 33.47%. This can help when a
stroke is weak or disconnected. However, the small white marks in the background also
become larger.

For example, a small gap separates S and e in the word Set in the binary input, but
dilation joins the two letters. This could make character separation more difficult. It
does not mean that a damaged stroke has been repaired.

Saved teaching view: figures/02_dilation.png

Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 3/6: Erosion: reduce the foreground
================================================================================

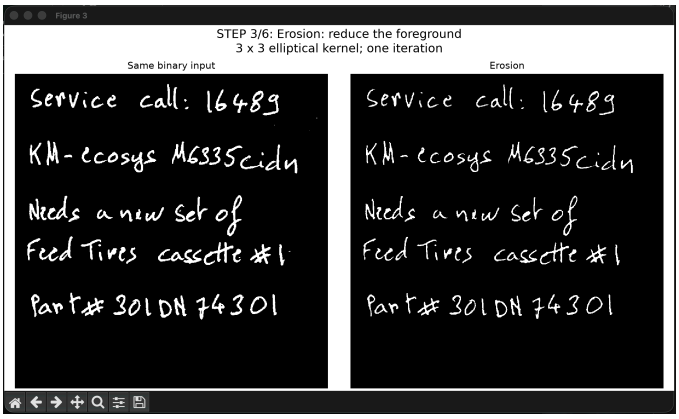
Erosion removes pixels around the white foreground boundaries, making the strokes
thinner. A pixel remains white only when every active kernel position fits inside the
white handwriting.

Erosion removes 11,806 foreground pixels, or 32.82% of the input foreground. Some small
background specks disappear, but thin portions of letters can also be removed.

In the model label, a thin curved stroke becomes broken near the following i. The
background may look cleaner in some areas, but part of the handwriting has been lost.
See figures/07_stroke_detail.png to compare the stroke before and after erosion.

Saved teaching view: figures/03_erosion.png

Enter C to continue or Q to quit, then press Enter:
```



```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 4/6: Opening: remove small foreground marks
================================================================================

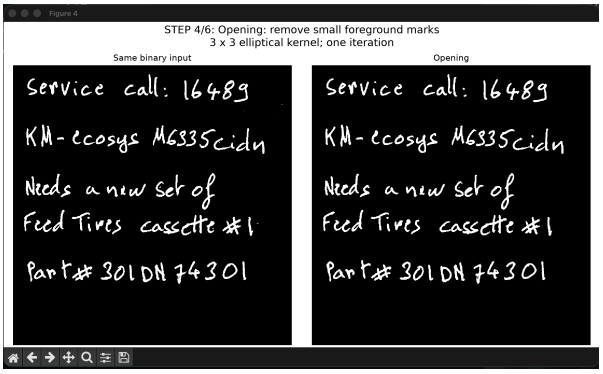
Opening applies erosion followed by dilation. Erosion removes small foreground
structures, and dilation expands the parts that remain. However, dilation cannot restore
a mark removed completely by erosion.

Opening removes 296 foreground pixels (0.82%) without adding white pixels. Three small
isolated marks near the upper-right background disappear. Most of the handwriting
remains close to its original stroke width.

The curved stroke in the model label stays connected after opening, unlike direct
erosion. Some other small fragments and boundary details are removed. A detached mark
could belong to a letter, so its removal does not automatically mean the writing has
been improved.

Saved teaching view: figures/04_opening.png

Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 5/6: Closing: fill small gaps
================================================================================

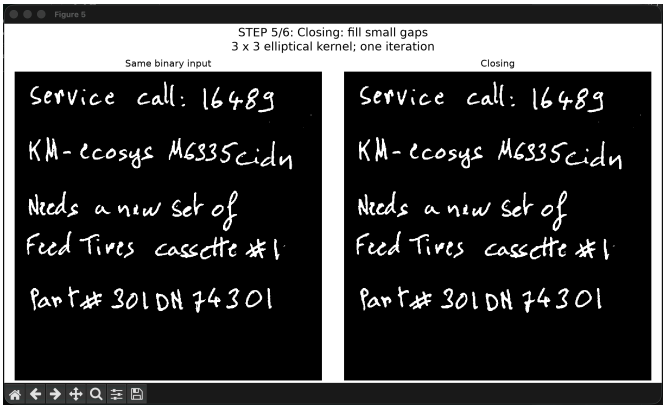
Closing applies dilation followed by erosion. Dilation can fill a small hole or gap,
while erosion reduces the expanded outer boundary. A connection made during dilation may
still remain afterward.

Closing adds 282 foreground pixels (0.78%) without removing foreground. The handwriting
changes very little overall, but the small isolated marks in the upper-right background
remain.

The S and e in Set become connected. This reduces the number of connected regions from
87 to 86. The program cannot determine whether this space was a damaged stroke or a
separation between letters. See figures/08_gap_detail.png to compare the gap.

Saved teaching view: figures/05_closing.png

Enter C to continue or Q to quit, then press Enter:
```



```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 6/6: Compare the four operations
================================================================================

The final comparison shows the straightened color note, its binary input, and the four
morphological results. All four start with the same binary image; they are not a
sequence of processing steps.

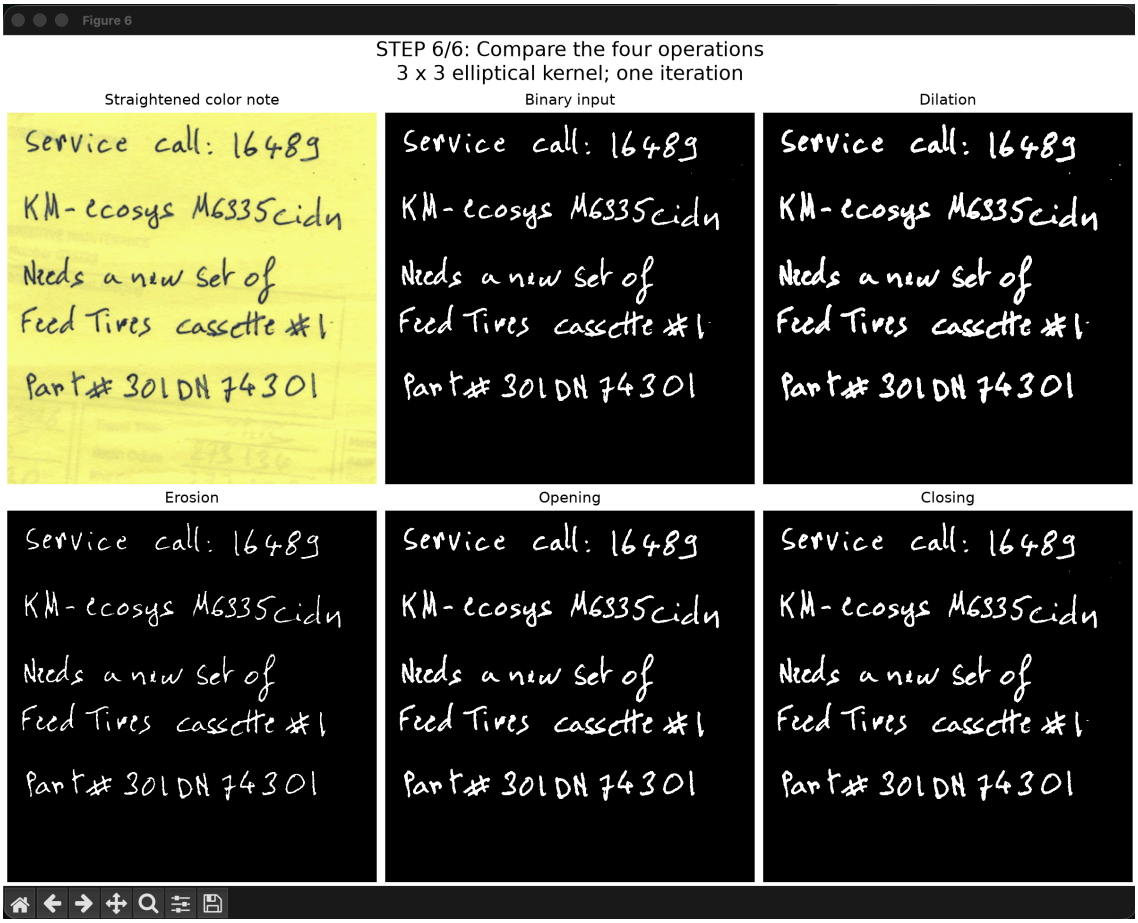
    Operation    Foreground     Added   Removed   Change %   Regions
    binary         35,968         0         0       0.00        87
    dilation       48,006    12,038         0      33.47        82
    erosion        24,162         0    11,806     -32.82        83
    opening        35,672         0       296      -0.82        81
    closing        36,250       282         0       0.78        86

The foreground counts show how many white pixels were added or removed. A connected
region can contain several letters or only one small mark. These measurements do not
count characters or indicate whether an OCR system can recognize the handwriting
accurately.

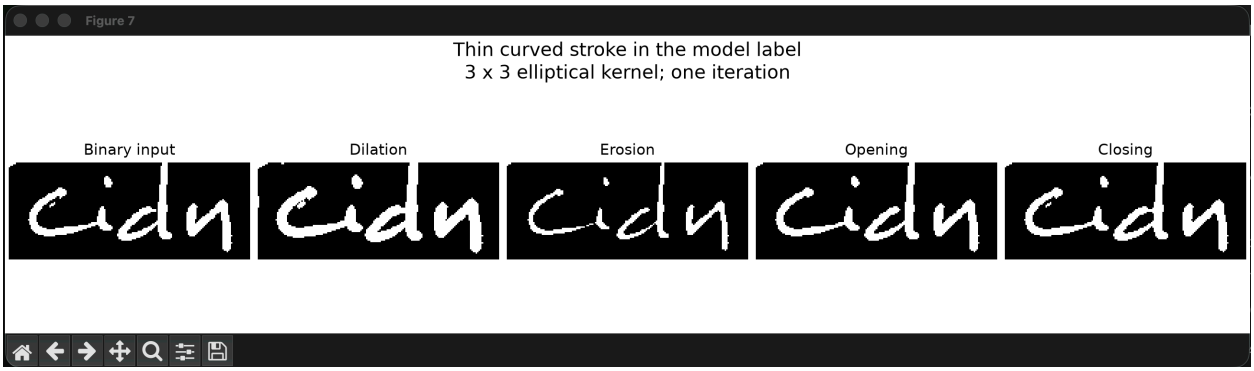
For this note, the 3 x 3 opening removes several background marks without the thicker
strokes of dilation or the broken curve seen with erosion. Closing changes fewer
foreground pixels, but it joins the space between S and e in Set. Much of the
improvement in paper contrast comes from Otsu thresholding before morphology. These
results do not mean opening will be best for every handwritten note.

Saved teaching view: figures/06_morphology_comparison.png

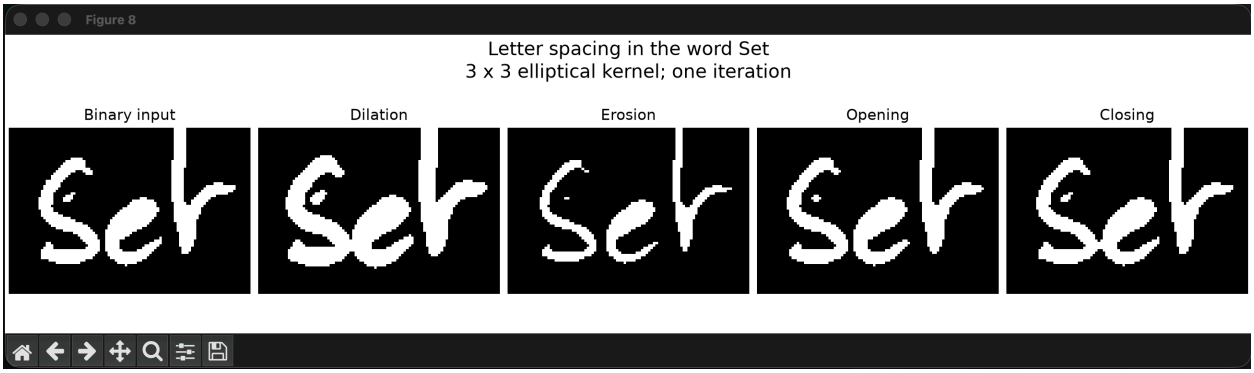
Enter C to continue or Q to quit, then press Enter:
```



```
Enter C to continue or Q to quit, then press Enter: c
STEP 6/6 - Close-up 1/2: figures/07_stroke_detail.png
Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c
STEP 6/6 - Close-up 2/2: figures/08_gap_detail.png
Enter Q to finish, then press Enter:
```




```
Saved results: /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-5/mod_images
alex.omegapy@MacStudio CSC515-CTAs %
```

Note: The figure showcases the terminal outputs and the images generated using the Matplotlib library.